Auto-Encoding Variational Bayes (VAE)

ref	https://arxiv.org/pdf/1312.6114
type	예습과제
week	week3

예습_생성 모델, 변형 오토인코더

1. 생성 모델이란

생성 모델

- 생성 모델 : 주어진 데이터를 학습하여 **데이터 분포**를 따르는 유사한 데이터(새로운 데이터이지만, 기존 데이터의 특정을 추출해서 사용)를 생성하는 모델, 판별자 모델에서 추출한 조합을 이용

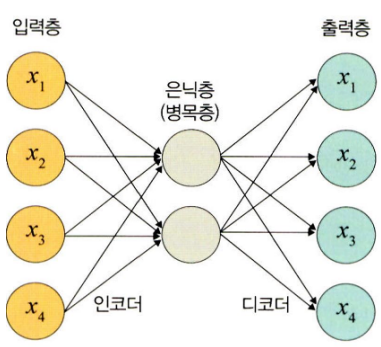
  (<>판별자 모델 : 입력에 대한 정답을 찾는 것)

- 생성 모델의 유형
  - 명시적 방법 → 변형 오토 인코더 : 확률 변수 p(x)를 정의해 사용 → 이미지의 latent space에서 샘플링, 완전히 새로운 이미지나 기존 이미지를 변형
  - 암시적 방법 → GAN : 확률 변수 p(x)를 정의하지 않고 p(x)를 sampling → 생성자와 판별자가 서로 경쟁, 가짜 이미지를 진짜 이미지와 최대한 비슷하게 만들도록 학습

2. 변형 오토인코더

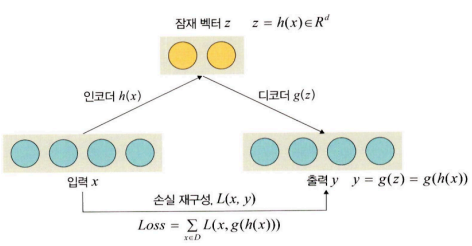
오토인코더란

: 단순히 입력을 출력으로 복사하는 신경망, 입력값보다 적은 hidden layer의 노드 수로 데이터를 가장 잘 표현하는 방법



1. encoder : 특성에 대한 학습 수행
2. hidden layer : 뉴런의 개수가 최소인 계층, 차원이 가장 낮은 입력 데이터의 압축 표현이 포함
3. decoder : gernerative network, hidden layer에서 압축 데이터를 원래대로 reconstruct
4. Reconstruction Loss : MLP와 동일한 구조의 오토인코더, 손실 함수는 입력과 출력의 차이를 계산

[공식으로 나타낸 encoder와 decoder]

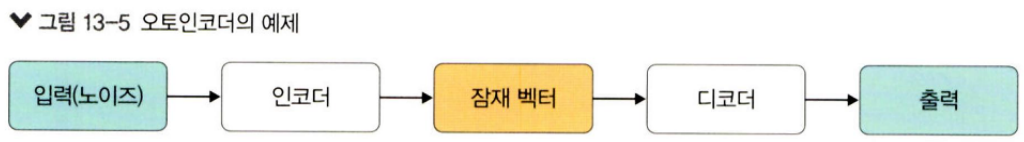


- representation vector z : (입력x와 출력 y는 같은 차원에 존재한다, 이때 encoder network를 통과하여 z를 얻는다) → z=h(x)
- output y : 입력 데이터와 크기가 같은 출력 데이터 y

  → y=g(z)=g(h(x))
- Loss : Loss=||x-y||^2

[오토인코더가 중요한 이유]

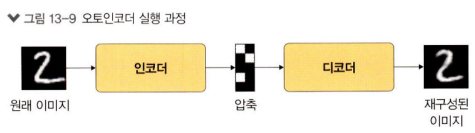
1. 데이터 압축 : 메모리 측면에서 장점
2. 차원의 저주 예방 : feature의 개수를 줄여주므로 데이터 차원 감소 → 예방
3. 특성 추출 : 비지도 학습으로 자동으로 중요한 feature를 찾아줌



변형 오토인코더

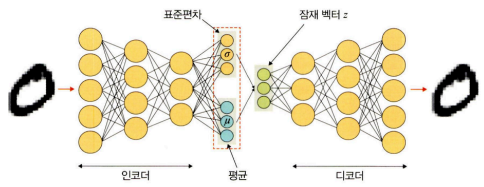
1. 오토인코더 vs 변형 오토인코더

[오토인코더]



- 차원 줄이기+실제 이미지와 동일한 이미지 출력

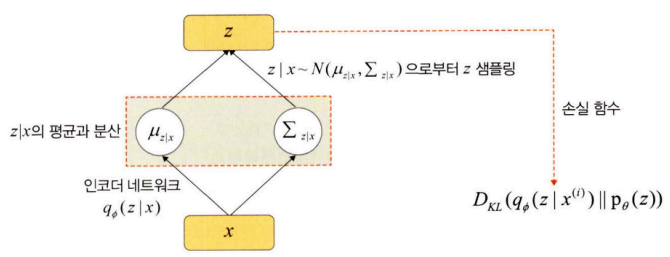
[변형 오토인코더]



- 표준편차와 평균을 이용하여 확률 분포를 만들고 샘플링 → decoder를 통과시켜 새로운 데이터 생성
- latent vector z를 활용 : 중요한 특성의 파라미터를 담도록 → 샘플링 후 분포의 오차를 이용하여 input과 다양한 데이터 생성

2. 변형 오토인코더의 encoder&decoder

[encoder network]



- encoder network : input x를 입력받아 latent vector z와 대응되는 평균과 분산을 구하는 네트워크

1)입력 x를 encoder network에 보내 평균과 분산 출력

2)1)의 값으로 2항에 대한 값(손실함수) 구하기

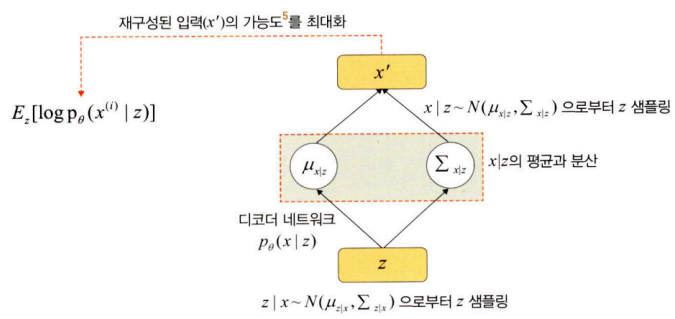
$$L(x^{(i)},\theta,\phi) = \underbrace{E_z[\log p_\theta(x^{(i)} \mid z)]}_{①} - \underbrace{D_{KL}(q_\phi(z \mid x^{(i)}) \,\|\, p_\theta(z))}_{②}$$

3)평균과 분산의 가우시안 분포에서 z를 샘플링

$$z \leftarrow z \mid x \sim N(\mu_{z|x}, \Sigma_{z|x})$$

⇒ z는 decoder 네트워크로 입력

[decoder network]



- decoder network : z를 입력받아 x와 대응되는 평균과 분산을 구하는 네트워크

1)샘플링한 z를 decoder network에 보내 1항의 값 구하기

$$L(x^{(i)},\theta,\phi) = \underbrace{E_z[\log p_\theta(x^{(i)} \mid z)]}_{①} - \underbrace{D_{KL}(q_\phi(z \mid x^{(i)}) \,\|\, p_\theta(z))}_{②}$$

2)평균과 분산의 가우시안 분포에서 z를 샘플링 후 x′ 구하기

$$x' \leftarrow x \mid z \sim N(\mu_{x|z},\ \Sigma_{x|z})$$

3)역전파를 활용 → likelihood가 증가하는 방향으로 기울기, 파라미터 업데이트

$$L(x^{(i)},\ \theta,\ \phi)$$

*encoder와 decoder에서 활용되는 수식

$$L(x^{(i)},\theta,\phi) = \underbrace{E_z[\log p_\theta(x^{(i)} \mid z)]}_{①} - \underbrace{D_{KL}(q_\phi(z \mid x^{(i)}) \,\|\, p_\theta(z))}_{②}$$

1항 : z가 주어졌을 때 x′를 표현하기 위한 확률밀도 함수 → decoder network

- **decoder network의 가능도**가 크면 θ가 데이터를 잘 표현한거
- 1항이 크면 모델의 가능도 커짐 → 가짜 데이터 잘 생성

2항 : x에서 z를 표현하는 확률밀도 함수 → encoder network와 가우시안 분포가 얼마나 유사한지

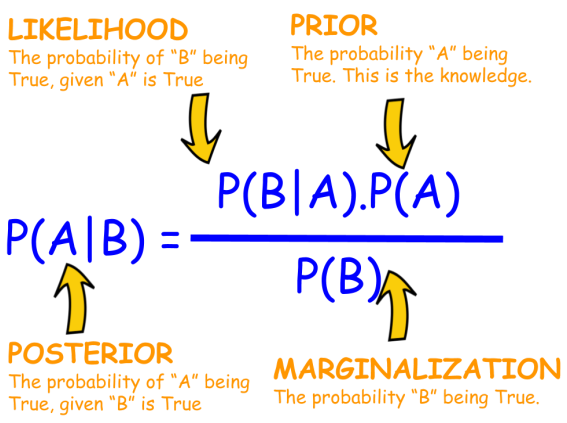
- 유사한 정도가 높을수록 Kullback–Leibler divergence 즉 KLD은 낮은 값을 나타냄 → encoder network가 가우시안 분포를 최대한 잘 표현할 수 있는 likelihood가 최대화
- 2항이 작을수록 모델 가능도 커짐 → 가짜 데이터 잘 생성

논문 리뷰

▼ Terminology

iid

: Independent and identically distribution : 독립 항등 분포 / Independent : 각각 사건이 다른 사건에 영향을 주지 않는 / identically distribution : 같은 확률분포를 가지는 ex) 주사위굴리기



posterior

: 사후확률, P(A|B) → 관측값(B)이 주어졌을 때, 구하고자 하는 대상(A)이 나올 확률

likelihood

: 구하고자 하는 대상(A)이 주어졌을 때, 관측값(B)이 나올 확률

variance

: 오차

variational

: 변분, 데이터를 간단하게 표현하면서도 원래 데이터의 중요한 특징을 잘 유지하도록 최적화하는 과정을 의미

→ VAE에서는 가장 적절한 간단한 분포를 찾기 위해 계속 조정을 갖는 것

*Abstract 참고

무엇이 문제인가? 연속적인 잠재변수 & large dataset에서 해결하기 어려운 사후 확률

무엇이 목표인가? 해당 문제에서 어떻게 방향성 확률 모델에서 효율적인 추론과 학습을 진행하는가

제안? stochastic varrational inference and learning algorithm - 큰 데이터셋에서 확장 가능, 계산 불가한 경우에도 milde한 미분 가능성 o

contribution?

- reparameterization of variational lower bound, SGD 방법론을 활용하여
- 위의 lower bound estimator를 활용하여 posterior 추론이 근사 추론 모델에 의해 풀 수 없는 posterior에 fit되는거

1. Introduction

💡 논문에서 다루고 있는 주제가 무엇인지와 해당 주제의 필요성이 무엇인가
논문에서 제안하는 방법이 기존 방법의 문제점에 대응되도록 제안 되었는가

논문이 다루는 분야

해결할 수 없는 사후 확률 분배(**intractable posterior distributions**)를 갖는 연속 잠재 변수(**continuous latent variables**)와 파라미터를 갖는 **directed probabilistic models**에서의 근사 추론 및 학습

해당 task에서 기존 연구 한계점

기존 VB에서 근사 사후확률의 general case에서의 해결이 어려움.

논문의 contributions

- **SGVB (Stochastic Gradient Variational Bayes) 추정기** 제안
  - variational lower bound의 재매개변수화를 활용하여 샘플링을 미분가능한 추정기로 만든다.
  - **continuous latent variables and/or param**을 갖는 posterior 추론의 근사에 활용됨
  - standard SGD로 최적화 진행
- **AEVB (Auto-Encoding Variational Bayes) 알고리즘** 제안
  - **SGVB** 추정기를 활용하여 인지 모델(encoder)을 최적화함으로 추론과 학습을 진행하는 알고리즘
  - encoder-decoder 구조의 NN에서 샘플링을 활용하여 사후 확률 근사를 진행

2. Related Work

💡 Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가
제안 방법의 차별성을 어떻게 표현하고 있는가

1. **wake-sleep algorithm**

- 공통점 : recog model로 posterior 추출 / AEVB에서 datapoint마다 연산 복잡도 비슷함
- 차이점 : 두 개의 목표함수 각각 최적화 필요 <> VAE : ELBO($\mathcal{L}$)만 최적화하면 됨

2. **Stochastic Variational Inference (SVI)**

- 공통점 : 확률적으로 lower bound를 추정하는 시도
- 차이점 : naive한 미분방식의 활용 → 오차가 너무 큼
  → VAE : 재매개변수화 트릭을 활용하여 이 오차를 줄이게 됨.

3. **autoencoders**

- 공통점 : lower bound를 최대화와 대응됨.

- 차이점 : 복원 오차(reconstruction criterion)를 단순히 줄일려고만 함 → 유용한 representations을 학습하지 못함. & 다양한 정규화 방법 요구

3. 제안 방법론

💡 Introduction에서 언급된 내용과 동일하게 작성되어 있는가
Introduction에서 언급한 제안 방법이 가지는 장점에 대한 근거가 있는가
제안 방법에 대한 설명이 구현 가능하도록 작성되어 있는가

Main Idea

- 추론하기 어려운 실제 사후 분포 $p_\theta(z|x)$ 대신 근사치 $q_\theta(z|x)$, recognition model(확률 encoder)
- 확률 decoder $p_\theta(x|z)$: code z가 주어졌을 때 x에 대응하는 가능값에 대한 분포
- ELBO : 실제 사후 분포 $p_\theta(z|x)$ 대신 "적어도 이것보다 성능 좋음"의 하한선을 찾아 최적화
  - 목표 : ELBO **$\mathcal{L}$**의 최대화가 필요
  - 문제 : 기존 샘플링 방식으로 encoder의 ϕ에 대하여 미분 불가
  - 해결 : z를 ㅁ분가능한 형태로 재정의하면서 역전파 가능하게
- Reparameterization trick : $z=\mu(x)+\sigma(x)\cdot\epsilon$, $\epsilon\sim N(0,1)$ 형태로 변환
  - 미분 가능한 형태로 바꾸면서 역전파를 가능하게 → 학습 가능
  - ϵ : sampling의 랜덤성을 챙기면서 미분이 가능하도록

Contribution

- SGVB Estimator : 미분 가능한 ELBO 추정기 계사니
- AEVB Algorithm: Encoder–Decoder 구조의 오토인코더, 모델 학습 루프
- Gaussian 분포 외에도 다양한 분포에 적용 가능 → 범용성
- 대규모 데이터셋에 적합한 mini-batch 학습 구조 제시.

4. 실험 및 결과

💡 Introduction에서 언급한 제안 방법의 장점을 검증하기 위한 실험이 있는가

Dataset

- MNIST
- Frey Face datasets

Baseline

- Wake-Sleep 알고리즘
- Monte Carlo EM(MCEM)

결과

- likelihood lower bound : latent space(Nz)를 늘려도 overfitting 발생이 없음
- Marginal likelihood : 매우 낮은 차원의 잠재 공간에서 AEVB&Wake Sleep method의 조합이 기존 MCEM&HMC만큼 효율을 보임
  - 작은 데이터셋 : Wake-Sleep, MCEM, AEVB 셋 다 어느정도 수렴 & AEVB의 우위
  - 큰 데이터셋 : AEVB 및 Wake-Sleep는 수렴, MCEM는 비효율적
- latent space(2D) : 데이터 구조를 시각화 ㄱㄴ

5. 결론 (배운점)

💡 연구의 의의 및 한계점, 본인이 생각하는 좋았던/아쉬웠던 점

의의

- 재매개변수화 트릭(Reparameterization Trick)의 발견: 미분 불가능한 샘플링을 미분 가능하게 바꾼다는 발상
- 학습의 효율성: 기존의 Wake-Sleep 같은 방식보다 훨씬 빠르고 안정적으로 학습이 가능하며, 대규모 데이터셋(Big Data)에도 적용할 수 있는 길을 열었음.

한계

- 잠재 변수 z → 가우시안 분포의 단순함, 현실에서는 데이터의 분포가 단순하지 않을텐데..,,

6. 헷갈리는거

[SGVB Estimator & AEVB Algorithm]

1. SGVB Estimator: 점수 계산기
- **SGVB (Stochastic Gradient Variational Bayes)**
Stochastic (확률적): 데이터를 다 안 쓰고 Mini batch로 쪼개서 본다.
Gradient (기울기): 미분을 해서 어디로 갈지 정한다.
Variational Bayes (변분 베이즈): $\mathcal{L}$(엘) 계산.
역할: 원래는 계산 불가능한 적분($\mathbb{E}$)을 '재매개변수화 기법($z = \mu + \sigma \epsilon$)'을 써서 컴퓨터가 미분 가능한 형태의 수식(식 7)으로 바꿔놓은 **'공식'** 그 자체. 즉, "미분 가능한 점수 계산 공식"이 SGVB이다.

→ **미분 가능한 하한선($\mathcal{L}$) 도출**

2. AEVB Algorithm: 전체 학습 루프
- **AEVB (Auto-Encoding Variational Bayes)**
Auto-Encoding: 인코더와 디코더가 있는 **오토인코더 구조**를 쓴다.
VB: 위에서 말한 **변분 베이즈($\mathcal{L}$)** 방식을 결합했다.
역할: SGVB라는 '계산기'를 들고 실제로 데이터를 넣어서 모델을 학습시키는 '전체 매뉴얼(논문의 Algorithm 1)'

Algorithm 전개도
1. 데이터를 뽑고(X^M),
2. 노이즈를 섞어서(ε),
3. SGVB 계산기로 점수($\mathcal{L}$)와 미분값(g)을 낸 뒤,
4. 파라미터를 업데이트하는 **일련의 반복 과정**을 통칭합니다.

→ 실제 파라미터(θ,ϕ) 최적화

⇒ SGVB는 미분 가능한 '수학 공식'이고, AEVB는 그 공식을 사용해 모델을 훈련시키는 '전체 알고리즘'

[KL 발산 공식을 활용하여 ELBO 공식 유도]

2.2 The variational bound ⇒ ELBO (Evidence Lower Bound) 설명

The marginal likelihood is composed of a sum over the marginal likelihoods of individual datapoints $\log p_{\boldsymbol{\theta}}(\mathbf{x}^{(1)}, \cdots, \mathbf{x}^{(N)}) = \sum_{i=1}^{N} \log p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$, which can each be rewritten as:

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = D_{KL}(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}^{(i)})||p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x}^{(i)})) + \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}^{(i)}) \tag{1}$$

The first RHS term is the KL divergence of the approximate from the true posterior. Since this KL-divergence is non-negative, the second RHS term $\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}^{(i)})$ is called the (variational) *lower bound* on the marginal likelihood of datapoint i, and can be written as:

하한선

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \geq \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}^{(i)}) = \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[-\log q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}) + \log p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})\right] \tag{2}$$

which can also be written as:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}^{(i)}) = -D_{KL}(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}^{(i)})||p_{\boldsymbol{\theta}}(\mathbf{z})) + \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}^{(i)})}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}|\mathbf{z})\right] \tag{3}$$

We want to differentiate and optimize the lower bound $\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}^{(i)})$ w.r.t. both the variational parameters $\boldsymbol{\phi}$ and generative parameters $\boldsymbol{\theta}$. However, the gradient of the lower bound w.r.t. $\boldsymbol{\phi}$ is a bit problematic. The usual (naïve) Monte Carlo gradient estimator for this type of problem is: $\nabla_{\boldsymbol{\phi}}\mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z})}[f(\mathbf{z})] = \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z})}\left[f(\mathbf{z})\nabla_{q_{\boldsymbol{\phi}}(\mathbf{z})}\log q_{\boldsymbol{\phi}}(\mathbf{z})\right] \simeq \frac{1}{L}\sum_{l=1}^{L} f(\mathbf{z})\nabla_{q_{\boldsymbol{\phi}}(\mathbf{z}^{(l)})}\log q_{\boldsymbol{\phi}}(\mathbf{z}^{(l)})$ where $\mathbf{z}^{(l)} \sim q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}^{(i)})$. This gradient estimator exhibits exhibits very high variance (see e.g. [BJP12]) and is impractical for our purposes.

*ELBO는 왜? : 진과 거의 data p_θ를 구하긴 어려움 → "적어도 이것은 내 성능 좋음"의 하한선을 찾아 최적화 (하한선을 올리도록)

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = \underbrace{D_{KL}(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}^{(i)})||p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x}^{(i)}))}_{\text{추측한 분포 } q\text{와 실제 정답 } p\text{가 얼마나 다른가}} + \underbrace{\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}^{(i)})}_{\text{ELBO}}$$

→ 항상 0 이상

전체점수 $\log p_\theta(x^i)$: 내가 맞힌 점수 $\mathcal{L}$ + $\underbrace{\text{실제 정답과의 오차 } D_{KL}}_{\geq 0}$ 이므로 전체점수 $\geq \mathcal{L}$ 이 성립

$\mathcal{L}$ ② 항

* KL 발산 : $D_{KL}(P||Q) = \int p(x) \log \frac{p(x)}{q(x)} dx$ → 통계에서 E는 연속확률변수에서는 적분

확률분포 P와 Q의 차이 측정 → 기댓값

- 식② → 에너지 안정, 식③ → VAE의 학습, KL 발산 (Regularization) ⊕ 재구성 오차 (복원) → 딥러닝 코드
- 베이즈 정리 : $p_\theta(x, z) = p_\theta(x|z)\, p_\theta(z)$

식② $\mathcal{L}(\theta, \phi; x^i) = E_{q_\phi(z|x)}\left[\log p_\theta(x|z) + \log p_\theta(z) - \log q_\phi(z|x)\right]$

$= E_{q_\phi(z|x)}(\log p_\theta(x|z)) - E_{q_\phi(z|x)}(\log q_\phi(z|x) - \log p_\theta(z))$

$= E_{q_\phi(z|x)}(\log p_\theta(x|z)) - D_{KL}[q_\phi(z|x) \,||\, p_\theta(z)]$